AM230 Neural Network Regression

Numerical Optimization Methods

Luke Catalano
UC Santa Cruz, M.S. Quantitative Economics

February 2026

Project Overview & Regression Setup

We consider a system whose behavior is described by an unknown input–output map

$$\mathcal{F} : \mathbb{R}^d \mapsto \mathbb{R}^p$$

The function $\mathcal{F}$ may represent a physical process, a computational routine, or any black-box mechanism for which only input–output measurements are available. We assume that $\mathcal{F}$ itself is not accessible. Instead, we observe a finite set of input–output samples:

$$\{(x^i, y^i)\}_{i=1}^N$$

Where $x^i \in \mathbb{R}^d$ denotes the input, where d is the dimension of the input feature space, and $y^i \in \mathbb{R}^p$ denotes the output, where p is the dimension of the output space.

The objective is to construct a data-driven model:

$$f(x) \approx \mathcal{F}(x)$$

using only the observed input–output pairs. Such a model can be used for prediction, simulation, or as a surrogate model in downstream analysis. In this project, we will use a neural network to construct our model:

$$\mathcal{F}(x) \approx f^{NN}(x; \theta)$$

where $f^{NN}(x; \theta)$ is a feedforward neural network with parameter θ. The parameter will be determined by minimizing a data-missfit objective. Neural networks can be viewed as a nonlinear generalization of linear regression, in which the basis functions themselves are parameterized and learned from data rather than fixed a priori.

Feedforward Neural Network Model

A feedforward neural network consists of the following components:

- An input layer, which receives the input vector $x \in \mathbb{R}^d$
- One or more hidden layers, which apply nonlinear transformations to input x or intermediate features.
- An output layer, which produces the final model output.

Hidden Layer Optimization Objective

For simplicity, we assume that each hidden layers contain the same number of neurons denoted as m. The first hidden layer takes input $x \in \mathbb{R}^d$ and generates an intermediate output $z^{(1)} \in \mathbb{R}$ according to:

$$s^{(1)} = W^{(1)}x + b^{(1)}, \quad z^{(1)} = \sigma(s^{(1)})$$

where $\sigma(\cdot)$ is a nonlinear function called an activation function, which is applied component wise to its argument. For any $v = [v_1, v_2, \ldots, v_m]^T$

$$\sigma(v) = \begin{bmatrix} \sigma(v_1) \\ \vdots \\ \sigma(v_m) \end{bmatrix}$$

$W^{(1)} \in \mathbb{R}^{m \times d}$ and $b^{(1)}$ are the weights and biases of the first hidden layer. They will be determined by minimizing the following loss function:

$$\min_{\theta \in \mathbb{R}^P} L(\theta) = \min_{\theta \in \mathbb{R}^P} \frac{1}{2N} \sum^{N} ||f^{NN}(x^i; \theta) - y^i||_2^2$$

where P is the total number of trainable parameters in the neural network.

The second hidden layer functions in the same way, but takes $z^{(1)}$ as input and generates an indermediate output $z^{(1)} \in \mathbb{R}^m$ according to

$$s^{(2)} = W^{(2)}z^{(1)} + b^{(2)}, \quad z^{(2)} = \sigma(s^{(2)})$$

where $W^{(2)} \in \mathbb{R}^{m \times m}$ and $b^{(2)} \in \mathbb{R}^m$ are the weights and biases of the second hidden layer. The same pattern applies for the remaining hidden layers for $n \geq 2$

$$s^{(n)} = W^{(n)}z^{(n-1)} + b^{(n)}, \quad z^{(n)} = \sigma(s^{(n)})$$

with trainable parameters $W^{(n)} \in \mathbb{R}^{m \times m}$ and $b^{(n)} \in \mathbb{R}^m$

Finally, the output layer maps $z^{(n)}$ to the neural network output

$$f^{NN}(x; \theta) = W^{(n+1)}z^{(n)} + b^{(n+1)}$$

where $W^{(n+1)} \in \mathbb{R}^{p \times m}$ and $b^{(n+1)} \in \mathbb{R}^p$ are the weights and biases of the output layer.

The feedforward neural network generates its model prediction through a composition of affine transformations and nonlinear activations, fully defined by the parameter vector

$$\theta = (W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, \ldots, W^{(n+1)}, b^{(n+1)})$$

together with the nonlinear activation function σ.

Problem size in neural network training

Total Trainable Parameters (1.1a)

First, we will compute the total number of trainable parameters (weights and biases) in terms of d, m, n, and p for a fully connected feedforward neural network as seen above.

Layer 1 (Input $\rightarrow$ Hidden Layer 1) contains the weights $m \times d$ and the bias m, providing $md + m$ trainable parameters.

Hidden layers 2 through n (Hidden Layer $i > 1 \rightarrow$ Hidden Layer $i + 1 < n + 1$) each contain weights $m \times m = m^2$, and bias m. There are n-1 of these layers, providing $(n-1)(m^2 + m)$ trainable parameters.

Output Layer (Hidden layer $n \rightarrow$ Output) contains the weights pm, and bias p, providing $pm + p$ trainable parameters.

Combining all layers together gives a total trainable parameter count of:

$$P = (md + m) + (n-1)(m^2 + m) + (pm + p) = (n-1)m^2 + m(d + n + p) + p$$

Scaling with m and n, holding d and p fixed (1.1b)

As width (m) grows, the dominant term is $(n-1)m^2$, which is **quadratic in m**

As depth (n) grows, the dominant term is $(n-1)m^2$, which is **linear in n but quadratic in m.**

The most significant driver of parameter growth is width (m) as it scales quadratically compared to the linear growth of depth (n).

Memory Usage Assuming 8 bytes per entry (1.1c)

Suppose we fix d = 10, m = 1000, n = 100, and p = 10 (total parameters $P \approx 9.912 \times 10^7$). This would result in the following memory usage:

- Parameter Vector $\theta : 8P \approx 8 \cdot 99,120,010 = 792,960,080$ bytes ≈ 0.793 GB
- Full Hessian $\nabla^2 L(\theta) : 8P^2 \approx 8(9.912 \times 10^7)^2 \approx 8 \times 10^{15}$ bytes ≈ 80 PB

Storing θ is feasible ($O(P)$ memory), while storing a full Hessian is completely infeasible at this scale ($O(P^2)$ memory)

Implications for First-Order methods versus Second-Order Methods (1.1d)

We see firsthand that storing a full hessian requires petabytes worth of storage, making training/testing completely infeasible at this scale. Methods that require the derivation of the full hessian such as Newton's method and derivations such as BFGS, are impractical. Subsequently, we move to rely on first-order methods, which only require the computation and storage of the parameter set (1st order gradient), or limited memory BFGS, which relies on sparse approximations of the full Hessian.

Properties of Neural Network Regression

Nonconvexity of Neural Network Training (2.1a)

We consider the simplified neural network with one hidden layer ($n = 1$), one neuron ($m = 1$), input $x \in \mathbb{R}$, and output $\hat{y} \in \mathbb{R}$:

$$\hat{y} = w^{(2)} \tanh(w^{(1)}x + b^{(1)}) + b^{(2)}.$$

Define the residual

$$r(\theta; x, y) = \hat{y} - y = w^{(2)} \tanh(w^{(1)}x + b^{(1)}) + b^{(2)} - y,$$

so that the per-sample loss is

$$\ell(\theta; x, y) = \frac{1}{2} r(\theta; x, y)^2.$$

By the chain rule,

$$\nabla \ell(\theta) = r(\theta) \nabla r(\theta).$$

We now compute $\nabla r(\theta)$ component-wise. We define

$$s^{(1)} = w^{(1)}x + b^{(1)}, \quad z^{(1)} = \tanh(s^{(1)}).$$

Using $\tanh'(s) = 1 - \tanh^2(s)$, we compute:

$$\frac{\partial r}{\partial w^{(1)}} = w^{(2)}(1 - \tanh^2(s^{(1)}))x,$$

$$\frac{\partial r}{\partial b^{(1)}} = w^{(2)}(1 - \tanh^2(s^{(1)})),$$

$$\frac{\partial r}{\partial w^{(2)}} = z^{(1)},$$

$$\frac{\partial r}{\partial b^{(2)}} = 1.$$

Thus,

$$\nabla r(\theta) = \begin{bmatrix} w^{(2)}(1 - \tanh^2(s^{(1)}))x \\ w^{(2)}(1 - \tanh^2(s^{(1)})) \\ z^{(1)} \\ 1 \end{bmatrix} \implies \nabla \ell(\theta) = r(\theta) \begin{bmatrix} w^{(2)}(1 - \tanh^2(s^{(1)}))x \\ w^{(2)}(1 - \tanh^2(s^{(1)})) \\ z^{(1)} \\ 1 \end{bmatrix}.$$

Hessian and Failure of Positive Semidefiniteness (2.1b)

The Hessian of the per-sample loss satisfies

$$\nabla^2 \ell(\theta) = \nabla r(\theta) \nabla r(\theta)^T + r(\theta) \nabla^2 r(\theta).$$

We evaluate this expression at the parameter values

$$w^{(1)} = 0, \quad b^{(1)} = 0, \quad w^{(2)} = 0.$$

At this point,

$$s^{(1)} = 0, \quad z^{(1)} = \tanh(0) = 0, \quad \tanh'(0) = 1.$$

The prediction becomes

$$\hat{y} = b^{(2)} \implies r(\theta) = b^{(2)} - y.$$

The gradient of the residual simplifies to

$$\nabla r(\theta) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \implies \nabla r(\theta) \nabla r(\theta)^T = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Using the given expression for $\nabla^2 r(\theta)$ and evaluating at $w^{(1)} = b^{(1)} = w^{(2)} = 0$ (with $z^{(1)} = 0$ and $\tanh''(0) = 0$), all entries vanish except:

$$\frac{\partial^2 r}{\partial w^{(1)} \partial w^{(2)}} = x, \quad \frac{\partial^2 r}{\partial b^{(1)} \partial w^{(2)}} = 1.$$

Thus,

$$\nabla^2 r(\theta) = \begin{bmatrix} 0 & 0 & x & 0 \\ 0 & 0 & 1 & 0 \\ x & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Therefore,

$$\nabla^2 \ell(\theta) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} + (b^{(2)} - y) \begin{bmatrix} 0 & 0 & x & 0 \\ 0 & 0 & 1 & 0 \\ x & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The second matrix contains both positive and negative eigenvalues whenever $b^{(2)} - y \neq 0$. Hence, for some parameter value $b^{(2)}$, the Hessian has a negative eigenvalue and is not positive semi definite.

Therefore, the per-sample loss $\ell(\theta)$ is nonconvex in θ.

Extension to the Empirical Loss (2.1c)

The empirical loss is

$$L(\theta) = \frac{1}{2N} \sum_{i=1}^{N} \left(f^{NN}(x^i; \theta) - y^i \right)^2 = \frac{1}{N} \sum_{i=1}^{N} \ell_i(\theta).$$

Its Hessian is

$$\nabla^2 L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla^2 \ell_i(\theta).$$

Since we have shown that for some sample (x^i, y^i) and some parameter value θ, the Hessian $\nabla^2 \ell_i(\theta)$ has a negative eigenvalue, the average Hessian $\nabla^2 L(\theta)$ also fails to be positive semidefinite at that parameter value.

Thus, the empirical loss $L(\theta)$ is nonconvex in θ.

Backpropagation and Gradient Computation (2.2)

We consider a feedforward neural network with two hidden layers ($n = 2$) and one neuron in each hidden layer ($m = 1$). The input is $x \in \mathbb{R}^d$ and the output is one-dimensional $y \in \mathbb{R}$. The per-sample loss is

$$\ell(\theta) = \frac{1}{2}\big(f^{NN}(x;\theta) - y\big)^2 = \frac{1}{2}(\hat{y} - y)^2.$$

Forward pass

The forward pass computes intermediate variables:

$$s^{(1)} = (w^{(1)})^T x + b^{(1)}, \qquad z^{(1)} = \sigma(s^{(1)}),$$
$$s^{(2)} = w^{(2)} z^{(1)} + b^{(2)}, \qquad z^{(2)} = \sigma(s^{(2)}),$$
$$\hat{y} = f^{NN}(x;\theta) = w^{(3)} z^{(2)} + b^{(3)}.$$

Backward pass (2.2a)

Define the sensitivity variables

$$\delta^{(3)} := \frac{\partial \ell}{\partial \hat{y}}, \qquad \delta^{(2)} := \frac{\partial \ell}{\partial s^{(2)}}, \qquad \delta^{(1)} := \frac{\partial \ell}{\partial s^{(1)}}.$$

First,

$$\delta^{(3)} = \frac{\partial}{\partial \hat{y}}\left(\frac{1}{2}(\hat{y} - y)^2\right) = \hat{y} - y.$$

Next, using $\hat{y} = w^{(3)} z^{(2)} + b^{(3)}$ and $z^{(2)} = \sigma(s^{(2)})$,

$$\delta^{(2)} = \frac{\partial \ell}{\partial s^{(2)}} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial s^{(2)}} = \delta^{(3)} \cdot w^{(3)} \cdot \sigma'(s^{(2)}).$$

Finally, using $s^{(2)} = w^{(2)} z^{(1)} + b^{(2)}$ and $z^{(1)} = \sigma(s^{(1)})$,

$$\delta^{(1)} = \frac{\partial \ell}{\partial s^{(1)}} = \frac{\partial \ell}{\partial s^{(2)}} \frac{\partial s^{(2)}}{\partial z^{(1)}} \frac{\partial z^{(1)}}{\partial s^{(1)}} = \delta^{(2)} \cdot w^{(2)} \cdot \sigma'(s^{(1)}).$$

Thus the recursive relations are:

$$\boxed{\delta^{(3)} = \hat{y} - y, \qquad \delta^{(2)} = \delta^{(3)} w^{(3)} \sigma'(s^{(2)}), \qquad \delta^{(1)} = \delta^{(2)} w^{(2)} \sigma'(s^{(1)}).}$$

Gradient with respect to parameters (2.2b)

We now compute each partial derivative using the sensitivities.
Layer 3 (output layer). Since $\hat{y} = w^{(3)} z^{(2)} + b^{(3)}$,

$$\frac{\partial \ell}{\partial w^{(3)}} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w^{(3)}} = \delta^{(3)} z^{(2)}, \qquad \frac{\partial \ell}{\partial b^{(3)}} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b^{(3)}} = \delta^{(3)}.$$

Layer 2. Since $s^{(2)} = w^{(2)} z^{(1)} + b^{(2)}$,

$$\frac{\partial \ell}{\partial w^{(2)}} = \frac{\partial \ell}{\partial s^{(2)}} \frac{\partial s^{(2)}}{\partial w^{(2)}} = \delta^{(2)} z^{(1)}, \qquad \frac{\partial \ell}{\partial b^{(2)}} = \frac{\partial \ell}{\partial s^{(2)}} \frac{\partial s^{(2)}}{\partial b^{(2)}} = \delta^{(2)}.$$

Layer 1. Since $s^{(1)} = (w^{(1)})^T x + b^{(1)}$ with $w^{(1)} \in \mathbb{R}^d$,

$$\frac{\partial \ell}{\partial w^{(1)}} = \frac{\partial \ell}{\partial s^{(1)}} \frac{\partial s^{(1)}}{\partial w^{(1)}} = \delta^{(1)} x, \qquad \frac{\partial \ell}{\partial b^{(1)}} = \frac{\partial \ell}{\partial s^{(1)}} \frac{\partial s^{(1)}}{\partial b^{(1)}} = \delta^{(1)}.$$

Collecting terms, the gradient is

$$\boxed{\nabla_\theta \ell = \begin{bmatrix} \frac{\partial \ell}{\partial w^{(1)}} \\ \frac{\partial \ell}{\partial b^{(1)}} \\ \frac{\partial \ell}{\partial w^{(2)}} \\ \frac{\partial \ell}{\partial b^{(2)}} \\ \frac{\partial \ell}{\partial w^{(3)}} \\ \frac{\partial \ell}{\partial b^{(3)}} \end{bmatrix} = \begin{bmatrix} \delta^{(1)} x \\ \delta^{(1)} \\ \delta^{(2)} z^{(1)} \\ \delta^{(2)} \\ \delta^{(3)} z^{(2)} \\ \delta^{(3)} \end{bmatrix}.}$$

Neural Network Training: Initialization

As an example for studying optimization properties of neural network regression, we implement a feedforward neural network with two hidden layers in the MATLAB code

`models/ffnn2h_objective.m`

to approximate the following two-dimensional target function on the following domain:

$$f(x) = sin(2\pi(x_1^2 + x_2^2)e^{-2(x_1^2+x_2^2)}, \quad -1 \leq x_1, x_2 \leq 1$$

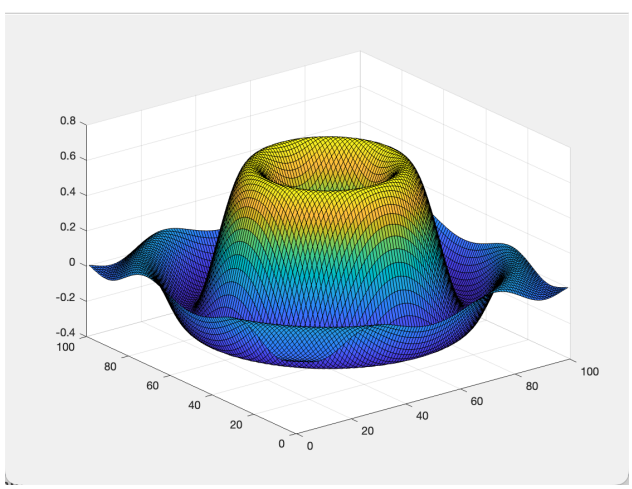


Figure 1: Target function f(x) on the domain $[-1, 1]^2$

Quadratic Regularization

To improve numerical conditioning of the resulting optimization problem, we introduce an L^2 (quadratic) regularization term into the loss function:

$$\bar{L} = \frac{1}{2N}\sum_{i=1}^{N}(f^{NN}(x^i;\theta) - y^i)^2 + \frac{\mu}{2}||\theta||_2^2$$

where $\mu > 0$ is our regularization parameter, and $||\theta||_2$ denotes the euclidean norm of the parameter vector. We set a sufficiently small quadratic penalty:

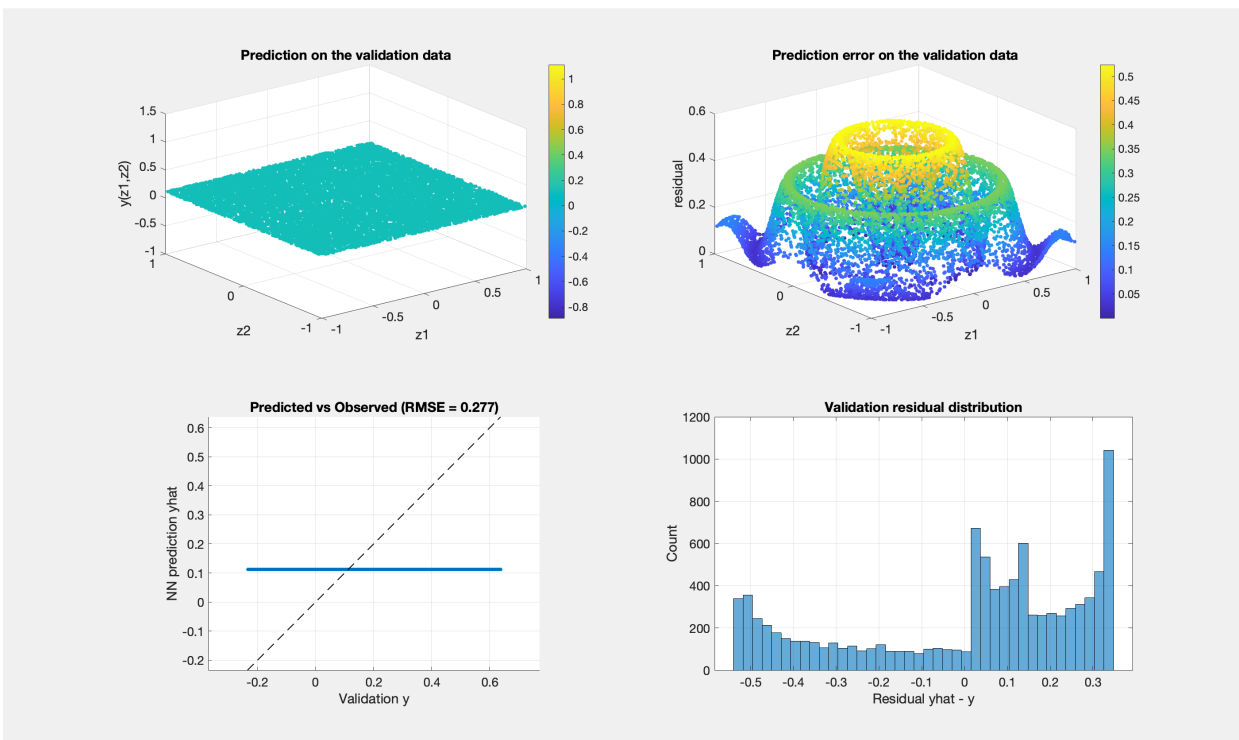
$$\mu = 10^{-6} = 0.000001$$

so that the loss function still captures the regression objective. Note this does not eliminate the non-convexity induced by the feed forward neural network architecture. It only contributes an additional linear term $\mu\theta$ to the gradient, so it does not alter the structure of back-propagation.

Initialization and stationary behavior (3.1)

We will apply gradient descent with line search to the neural network regression problem with $m = 10$, $n = 2$. We will study the solution from the zero initial condition ($\theta_0 = 0$), first numerically, then analytically.

Numerical Implementation (3.1a)



After 1 iteration of gradient descent with line search, the model appears to given up on approximating the data-generating process. Every prediction is equivalent to 0, and the model has not learned any patterns. RMSE: 0.2773 , MAE: 0.2341, AE: 0.5246

Analytical Derivation (3.1b)

To simplify the notation and focus on the core idea, assume $d = 1$ and $m = 1$. We can show that for any data set $\{(x^i, y^i)\}_{i=1}^N$, if the gradient descent algorithm is started from the zero initial condition

$$\theta_0 = \begin{bmatrix} w_0^{(1)} & b_0^{(1)} & w_0^{(2)} & b_0^{(2)} & w_0^{(3)} & b_0^{(3)} \end{bmatrix}^T = 0 \in \mathbb{R}^6$$

except possibly for the last parameter $b^{(3)}$, all other parameters remain equal to zero for all iterations k. Moreover, if the data set satisfies $\sum_{i=1}^N y^i = 0$, we can show that the numerical solution of gradient descent satisfies $\theta_k = 0$ for all iterations of k.

The network with $d = 1$ and $m = 1$ is defined as

$$f^{NN}(x;\theta) = w^{(3)}\sigma(w^{(2)}\sigma(w^{(1)}x + b^{(1)}) + b^{(2)}) + b^{(3)}$$

Given the starting parameters of $\theta_0 = 0 \in \mathbb{R}^6$, the initial output for any x is

$$f^{NN}(x^i;\theta) = 0\sigma(0\sigma(0x^i + 0) + 0) + 0 = 0$$

Gradient Descent follows the standard update procedure

$$\theta_{k+1} = \theta_k - \alpha p_k$$

With the gradient p_k defined for the squared error loss

$$\ell = \frac{1}{2}\sum_{i=1}^N (f^{NN}(x^i;\theta) - y^i)^2 = \frac{1}{2}\sum_{i=1}^N (w^{(3)}\sigma(w^{(2)}\sigma(w^{(1)}x + b^{(1)}) + b^{(2)}) + b^{(3)} - y^i)^2$$

Taking the partial derivative with respect to the output parameter $w^{(3)}$:

$$\frac{\partial \ell}{\partial w^{(3)}} = \frac{2}{2}\sum_{i=1}^N (w^{(3)}\sigma(w^{(2)}\sigma(w^{(1)}x + b^{(1)}) + b^{(2)}) + b^{(3)} - y^i) \times \sigma(z^{(2)})$$

$$\frac{\partial \ell}{\partial w^{(3)}} = \sum_{i=1}^N (f^{NN}(x^i;\theta) - y^i) \times \sigma(z^{(2)})$$

At $\theta_0 = 0$, $z^{(2)} = 0 \implies w^{(3)} = 0$. By the chain rule, during every iteration, every parameter (except $b^{(3)}$, as we are adding it, not multiplying in $f^{NN}(x^i;\theta)$) is multiplied by $w^{(3)}$, thus, we have that for all iterations k, all parameters excluding $b^{(3)}$ equal 0.

To show $\sum_{i=1}^N y^i = 0, \implies \theta_k = 0$ for all iterations of k, we take the gradient of the loss function:

$$\nabla \ell(\theta) = \frac{d}{d\theta}\left[\frac{1}{2}\sum_{i=1}^N (f^{NN}(x^i;\theta) - y^i)^2\right] = \sum_{i=1}^N (f^{NN}(x^i;\theta) - y^i)$$

We are given that the data satisfies $\sum_{i=1}^N y^i = 0$. We can substitute this fact into the gradient.

$$\sum_{i=1}^N (f^{NN}(x^i;\theta) - 0) = \sum_{i=1}^N (f^{NN}(x^i;\theta)) = \sum_{i=1}^N (0\sigma(0\sigma(0x + 0) + 0) + 0) = 0 \qquad \texttt{(def. 3.2b)}$$

Iterative Proof by Induction (3.b continued)

Let θ be a vector of parameters $\begin{bmatrix} w_0^{(1)} & b_0^{(1)} & w_0^{(2)} & b_0^{(2)} & w_0^{(3)} & b_0^{(3)} \end{bmatrix}^T$. It can be shown that when $\sum_{i=1}^{N} y^i = 0$, $\theta_k = 0$ for all iterations of k.

We define our initialization of the parameter vector as $\theta_0 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^T$.

and define our update per iteration as $\theta_{k+1} = \theta_k - \alpha \nabla \ell(\theta_k)$.

Initialization step (k = 0):

By construction, our intialization step is manually set equal to 0.

Base step (k = 1):

By `def. 3.2b`, we've shown that the gradient of the loss function $\nabla \ell$ is equivalent to 0 when $\sum_{i=1}^{N} y^i = 0$ and $\theta_0 = 0$. Therefore

$$\theta_1 = \theta_0 - \alpha \nabla \ell(\theta_0) = 0 - \alpha(0) = 0.$$

Inductive Step (k+1):

Consider the next iteration k+1, where the prior iteration k resulted in $\theta_k = 0$.

$$\theta_{k+1} = \theta_k + \alpha \nabla \ell(\theta_k) = 0 + \alpha(0) = 0$$

By induction, we have shown when $\sum_{i=1}^{N} y^i = 0$, $\theta_k = 0$ for all iterations of k. $\square$

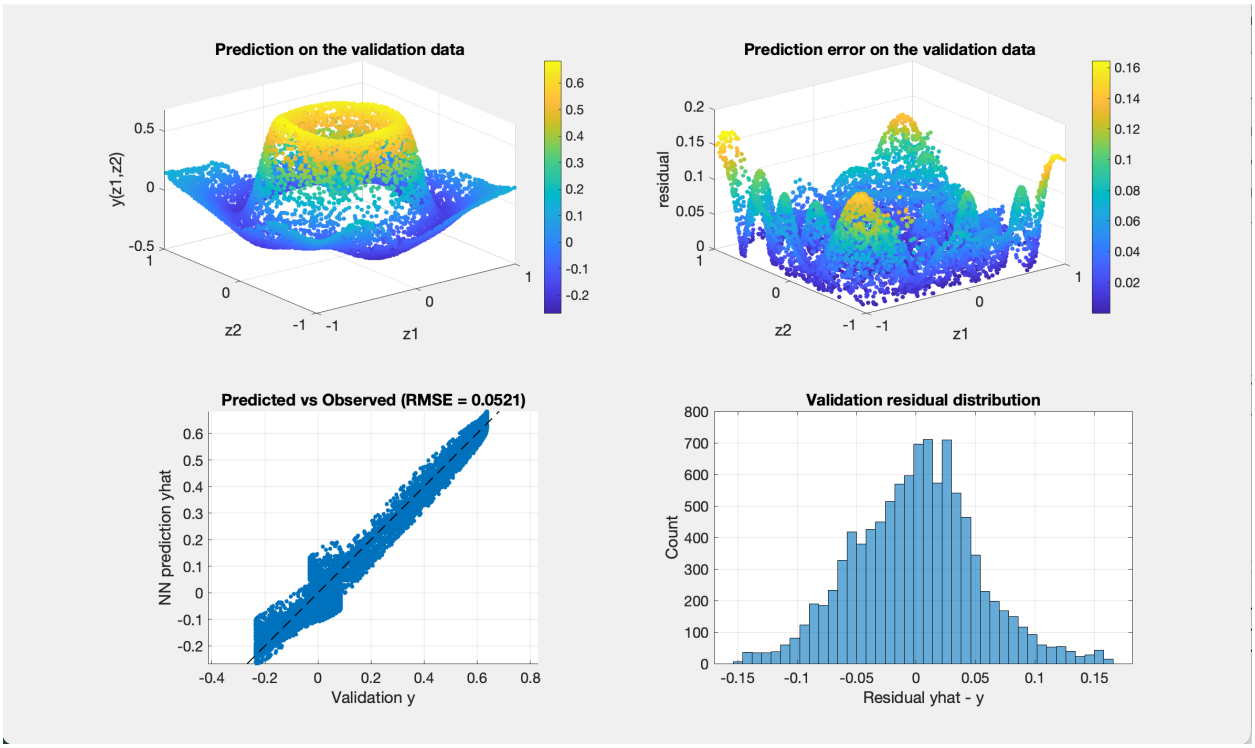
This phenomenon is a special case of symmetry-induced degeneracy. If all weights in a layer are initialized identically (in particular, to zero), the gradients of those weights remain identical throughout training, preventing the network from breaking symmetry and learning distinct features.

Initialization With Gaussian or Uniform Perturbations (3.2)

In practice, neural networks are initialized with small random perturbations; randomized shocks, whose values are drawn from a normal/uniform distribution, to introduce randomness into the initialization process.

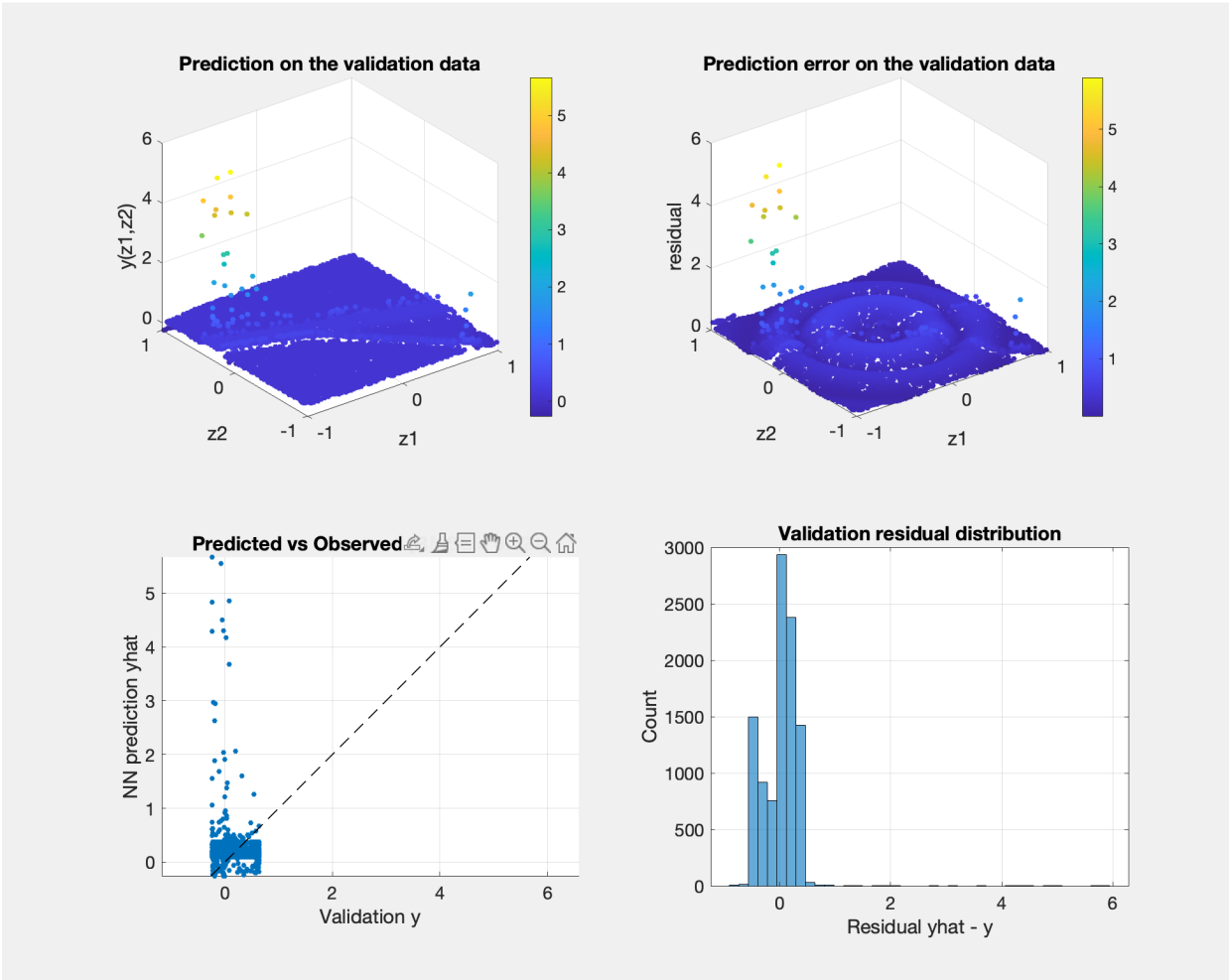
Numerical Implementation (3.2a)

```
stopping criterion opts.tolGrad = 5e-5, and run_GD = true,
theta0 = 0.1*randn(theta_len,1);
```



After 50,000 iterations of gradient descent with line search, the model appears to have approximated the data-generating process quite well. While this approximation is not perfect, the predictions on the validation data follow a functional form similar to the original function. Residuals $(\hat{y} - y)$ are approximately normally distributed, suggesting our model captures the main underlying patterns, leaving random, independent errors. RMSE: 0.0521, MAE: 0.0407, AE: 0.1645

```
stopping criterion opts.tolGrad = 5e-5, and run_GD = true,
theta0 = 10*randn(theta_len,1);
```



After 50,000 iterations of gradient descent with line search, the model has failed to approximate the data-generating process. We have achieved vastly different results from the prior experiment with a smaller initialization vector, and have a much larger variance in residuals. RMSE: 0.3212, MAE: 0.2354, AE: 5.9014

Initialization Scale Selection Choice (3.2b)

When the initialization scale is large, the gradient becomes almost zero during back propagation, and the model learns nothing. Conversely, when the scale is small, the gradient remains large enough during back propogation to learn patterns without diminishing too quickly.